```
# Q1- FOR FINDING THE INITIAL ACCELERATION
t = [0, 5, 10, 15, 20]
v = [0, 3, 14, 69, 228]
h = t[1] - t[0]
delta_v = [v[i + 1] - v[i] for i in range(len(v) - 1)]
initial_acceleration = delta_v[0] / h
print(f"Initial acceleration: {initial_acceleration} m/s²")

#Q2- FINDING THE FIRST DERIVATIVE AT X=1.2
x = [1.0, 1.2, 1.4, 1.6, 1.8, 2.0]
f = [0, 0.128, 0.544, 1.296, 2.432, 4.00]
h = x[1] - x[0]
delta_f = [f[i + 1] - f[i] for i in range(len(f) - 1)]
delta2_f = [delta_f[i + 1] - delta_f[i] for i in range(len(delta_f) - 1)]
delta3_f = [delta2_f[i + 1] - delta2_f[i] for i in range(len(delta2_f) - 1)]
dy_dx_at_1_2 = (
    delta_f[1] / h
    - delta2_f[1] / (2 * h)
    + delta3_f[1] / (6 * h)
)
print(f"First derivative at x = 1.2: {dy_dx_at_1_2}")

#Q3-FINDING THE FIRST DERIVATIVE AT X=0.4
x1 = [0.1, 0.2, 0.3, 0.4]
```

```
f_x1 = [1.10517, 1.22140, 1.34986, 1.49182]

h1 = x1[1] - x1[0]

derivative_at_0_4 = (f_x1[3] - f_x1[2]) / h1

print(f"First derivative of f(x) at x = 0.4 is approximately:
{derivative_at_0_4}")
```

```
#Q4-FINDING VALUES AT X=2.03

x2 = [1.96, 1.98, 2.00, 2.02, 2.04]

y2 = [0.7825, 0.7739, 0.7651, 0.7563, 0.7473]

h2 = x2[1] - x2[0]

index_closest_to_2_03 = 4

dy_dx_at_2_03 = (y2[4] - y2[3]) / h2

print(f"First derivative of y(x) near x = 2.03 is approximately:
{dy_dx_at_2_03}")
```

```
#Q5-EVALUATING THE NUMERICAL USING SIMPSONS 1/3 RULE

import numpy as np

def f(x):
    return np.exp(x) / (1 + x)

def simpsons_one_third(a, b, n):
    if n % 2 != 0:
        raise ValueError("Number of subintervals (n) must be even.")


    h = (b - a) / n
```

```
    x = np.linspace(a, b, n + 1)
    y = f(x)
    integral = y[0] + y[-1]
    integral += 4 * sum(y[1:-1:2])
    integral += 2 * sum(y[2:-1:2])
    integral *= h / 3
    return integral
a = 0
b = 6
n = 6
result = simpsons_one_third(a, b, n)
print(f"The value of the integral is approximately: {result}")

#Q6-EVALUATING THE NUMERICAL USING NUMERICAL METHODS
import numpy as np
from scipy.integrate import quad
def f(x):
    return 1 / (2 + x)

# Trapezoidal Rule
def trapezoidal_rule(a, b, n):
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = f(x)
```

```
    integral = h * (0.5 * y[0] + 0.5 * y[-1] + sum(y[1:-1]))
    return integral

# Simpson's 1/3 Rule
def simpsons_one_third(a, b, n):
    if n % 2 != 0:
        raise ValueError("Number of subintervals (n) must be even for Simpson's 1/3 Rule.")
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = f(x)
    integral = y[0] + y[-1] + 4 * sum(y[1:-1:2]) + 2 * sum(y[2:-1:2])
    return (h / 3) * integral

# Simpson's 3/8 Rule
def simpsons_three_eighth(a, b, n):
    if n % 3 != 0:
        raise ValueError("Number of subintervals (n) must be a multiple of 3 for Simpson's 3/8 Rule.")
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = f(x)
    integral = y[0] + y[-1] + 3 * sum(y[1:-1:3]) + 3 * sum(y[2:-1:3]) + 2 * sum(y[3:-1:3])
    return (3 * h / 8) * integral
```

```
# Weddle's Rule
def weddles_rule(a, b, n):
    if n % 6 != 0:
        raise ValueError("Number of subintervals (n) must be a multiple of 6 for Weddle's Rule.")
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = f(x)
    integral = 0
    for i in range(0, n, 6):
        integral += (
            (3 * h / 10)
            * (
                y[i]
                + 5 * y[i + 1]
                + y[i + 2]
                + 6 * y[i + 3]
                + y[i + 4]
                + 5 * y[i + 5]
                + y[i + 6]
            )
        )
    return integral
```

```
a, b = 0, 6
n = 6
actual_value, _ = quad(f, a, b)
trapezoidal = trapezoidal_rule(a, b, n)
simpsons_1_3 = simpsons_one_third(a, b, n)
simpsons_3_8 = simpsons_three_eighth(a, b, n)
weddles = weddles_rule(a, b, n)

print(f"Actual value: {actual_value:.6f}")
print(f"Trapezoidal Rule: {trapezoidal:.6f}")
print(f"Simpson's 1/3 Rule: {simpsons_1_3:.6f}")
print(f"Simpson's 3/8 Rule: {simpsons_3_8:.6f}")
print(f"Weddle's Rule: {weddles:.6f}")
```

REGISTRATION NUMBER=24BCE10683